Git and GitHub – Notes

1. What is Git?

Git is a **Distributed Version Control System (DVCS)** used to track changes in source code during software development. It helps developers manage different versions of a project, collaborate with others, and restore previous versions when needed.

FEATURES

- Tracks file changes
- Fast and lightweight
- Supports branching and merging
- Works offline
- Easy collaboration
- Open source

ADVANTAGES

- Maintains project history
- Prevents accidental data loss
- Enables teamwork
- Makes experimentation safe through branches

---

2. What is GitHub?

GitHub is a **cloud-based platform** that hosts Git repositories. It provides collaboration features such as pull requests, issue tracking, project management, and code reviews.

USES

- Store projects online
- Collaborate with teams
- Showcase projects in a portfolio
- Backup source code
- Contribute to open-source projects

---

3. Git vs GitHub

Git	GitHub

Version control software	Cloud hosting platform
Installed on your computer	Accessed through the web
Tracks file changes	Stores Git repositories online
Can work offline	Requires internet for syncing

---

4. Installing Git

WINDOWS

1. Download Git.
2. Run the installer.
3. Keep default settings.
4. Finish installation.

VERIFY INSTALLATION

```
git --version
```

---

5. Configure Git

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Check configuration:

```
git config --list
```

---

6. Git Repository

A repository (repo) is a folder that contains your project files along with Git's version history.

TYPES

- Local Repository
- Remote Repository

Initialize a repository:

```
git init
```

---

7. Git Workflow

Working Directory

↓

Staging Area

↓

Local Repository

↓

Remote Repository (GitHub)

---

8. Important Git Commands

CHECK STATUS

```
git status
```

Shows modified, staged, and untracked files.

ADD FILES

Add one file:

```
git add filename
```

Add all files:

```
git add .
```

COMMIT CHANGES

```
git commit -m "Initial commit"
```

A commit is a saved snapshot of your project.

VIEW COMMIT HISTORY

```
git log
```

Compact format:

```
git log --oneline
```

REMOVE FILE

```
git rm filename
```

RENAME FILE

```
git mv oldname newname
```

---

9. Branching

A branch is an independent line of development.

Create branch:

```
git branch feature
```

Switch branch:

```
git checkout feature
```

Create and switch:

```
git checkout -b feature
```

List branches:

```
git branch
```

Delete branch:

```
git branch -d feature
```

---

10. Merging

Merge another branch into the current branch.

```
git merge feature
```

Purpose:

- Combine work
- Integrate new features
- Finish development

---

11. GitHub Repository

Steps:

1. Create GitHub account.
2. Click **New Repository**.
3. Enter repository name.
4. Choose Public or Private.
5. Create repository.

---

12. Connect Local Project to GitHub

```
git remote add origin https://github.com/username/repository.git
```

Verify remote:

```
git remote -v
```

---

13. Push Code to GitHub

First push:

```
git push -u origin main
```

Next pushes:

```
git push
```

---

14. Clone Repository

Download an existing repository.

```
git clone repository-url
```

Example:

```
git clone https://github.com/user/project.git
```

---

15. Pull Latest Changes

```
git pull origin main
```

Downloads and merges the latest updates.

---

16. Fetch

```
git fetch
```

Downloads updates without merging.

---

17. Difference Between Pull and Fetch

Pull	**Fetch**
Downloads and merges	Downloads only
Updates current branch	Requires manual merge
Used for quick updates	Used to inspect changes first

---

18. Git Ignore

Ignore unnecessary files.

Create:

```
.gitignore
```

Example:

```
node_modules/
.env
*.log
dist/
```

---

19. Undo Changes

Discard changes:

```
git restore filename
```

Unstage file:

```
git restore --staged filename
```

Reset previous commit:

```
git reset --soft HEAD~1
```

---

20. Tags

Create tag:

```
git tag v1.0
```

Push tag:

```
git push origin v1.0
```

---

21. Fork

A fork is your own copy of another person's GitHub repository.

Purpose:

- Contribute to open source
- Experiment safely
- Keep personal changes separate

---

22. Pull Request (PR)

A Pull Request requests that your changes be reviewed and merged into another branch.

Steps:

1. Fork repository.
2. Clone fork.
3. Create branch.
4. Make changes.
5. Commit.
6. Push.
7. Open Pull Request.
8. Review.
9. Merge.

---

23. Merge Conflict

Occurs when Git cannot automatically combine changes.

Resolve by:

- Open conflicted files.
- Edit manually.
- Save.
- Add files.
- Commit again.

---

24. Common Git Commands Cheat Sheet

```
git init
git clone URL
git status
git add .
git commit -m "message"
git log
```

```
git branch
git checkout branch
git checkout -b branch
git merge branch
git pull
git push
git fetch
git remote -v
git restore file
git reset --soft HEAD~1
```

---

25. Best Practices

- Commit frequently.
- Write meaningful commit messages.
- Pull before pushing.
- Use branches for new features.
- Keep `.gitignore` updated.
- Never commit passwords or API keys.
- Review code before merging.

---

26. Quick Revision

- Git = Version Control System.
- GitHub = Online repository hosting platform.
- Repository = Project folder managed by Git.
- Commit = Snapshot of project changes.
- Branch = Independent development line.
- Merge = Combine branches.
- Clone = Download repository.
- Push = Upload changes.
- Pull = Download and merge updates.
- Fetch = Download updates only.
- Fork = Personal copy of someone else's repository.
- Pull Request = Request to merge changes.